

Assignment 2

Exercise groups 10

Magnus Fidje, Oskar Støre and Håkon Skramstad

Exercise 1

General solution of the form, using Master Theorem:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n), \quad a \geq 1, b > 1$$

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & \text{if } f(n) = O(n^{\log_b a - \varepsilon}) \text{ for some } \varepsilon > 0 & \text{(Case 1)} \\ \Theta(n^{\log_b a} \log n) & \text{if } f(n) = \Theta(n^{\log_b a}) & \text{(Case 2)} \\ \Theta(f(n)) & \text{if } f(n) = \Omega(n^{\log_b a + \varepsilon}) \text{ for some } \varepsilon > 0 & \text{(Case 3)} \\ & \text{and } a f(n/b) \leq c f(n) \text{ for some } c < 1 \end{cases}$$

a)

$$T(n) = 2T\left(\frac{n}{4}\right) + 1$$

$$a = 2, \quad b = 4, \quad f(n) = 1, \quad \log_b a = \log_4 2 = \frac{1}{2}$$

We check Case 1: $f(n) = 1 = O(n^{1/2 - \varepsilon})$ with $\varepsilon = \frac{1}{2}$, since $n^0 = O(n^0) = O(1)$.

$$T(n) = \Theta(n^{\log_4 2}) = \Theta(\sqrt{n})$$

b)

$$T(n) = 2T\left(\frac{n}{4}\right) + \sqrt{n}$$

$$a = 2, \quad b = 4, \quad f(n) = \sqrt{n}, \quad \log_b a = \log_4 2 = \frac{1}{2}$$

We check Case 2: $f(n) = \sqrt{n} = \Theta(n^{1/2}) = \Theta(n^{\log_b a})$.

$$T(n) = \Theta(\sqrt{n} \log n)$$

c)

$$T(n) = 2T\left(\frac{n}{4}\right) + n$$

$$a = 2, \quad b = 4, \quad f(n) = n, \quad \log_b a = \log_4 2 = \frac{1}{2}$$

We check Case 3: $f(n) = n = \Omega(n^{1/2+\varepsilon})$ with $\varepsilon = \frac{1}{2}$.

Regularity condition: $a f(n/b) = 2 \cdot \frac{n}{4} = \frac{n}{2} \leq c n$ with $c = \frac{1}{2} < 1$.

$$T(n) = \Theta(n)$$

d)

$$T(n) = 2T\left(\frac{n}{4}\right) + n^2$$

$$a = 2, \quad b = 4, \quad f(n) = n^2, \quad \log_b a = \log_4 2 = \frac{1}{2}$$

We check Case 3: $f(n) = n^2 = \Omega(n^{1/2+\varepsilon})$ with $\varepsilon = \frac{3}{2}$.

Regularity condition: $a f(n/b) = 2 \cdot \left(\frac{n}{4}\right)^2 = \frac{n^2}{8} \leq c n^2$ with $c = \frac{1}{8} < 1$.

$$T(n) = \Theta(n^2)$$

Exercise 2

```
Permute-Without-Identity(A)
  n = A.length
  for i = 1 to n-1
    swap A[i] with A[Random(i+1, n)]
```

The algorithm will never return the original ordering. That is because in the first step $i=1$, and it's swapped with some $A[j]$, where j is from 2 to n element. Since all elements are unique, then $A[1]$ will always be a different value than the original.

The algorithm will not generate all possible permutations for a given array. At iteration i , the algorithm will choose from $n-1$ positions. The total number of outcomes is:

$$(n-1) \times (n-2) \times \dots \times 1 = (n-1)!$$

Example: $i = 1$: swap $A[1]$ with $A[\text{Random}(2, 3)]$ will be one of these choices:

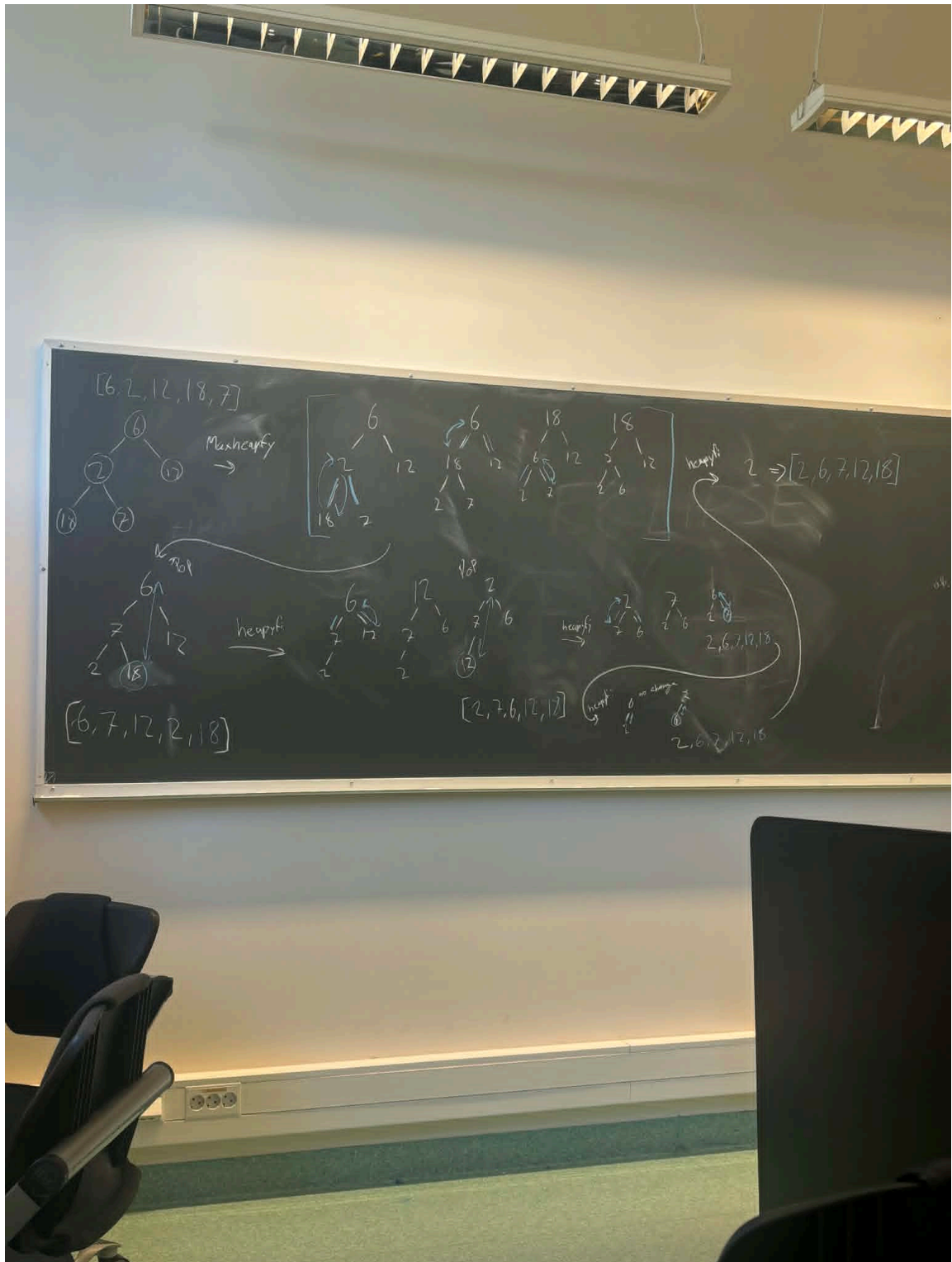
- $j = 2 \rightarrow [2, 1, 3]$
- $j = 3 \rightarrow [3, 2, 1]$

$i = 2$: swap $A[2]$ with $A[\text{Random}(3, 3)]$ that it's only once choice:

- $[2, 1, 3] \rightarrow [2, 3, 1]$
- $[3, 2, 1] \rightarrow [3, 1, 2]$

The algorithm produces 2 outcomes, each with probability 1/2

Exercise 3



Exercise 4

```

In [1]: def heapify(arr, i, n):
    smallest = i
    l = 2 * i + 1
    r = 2 * i + 2

    # If left child exists and is smaller than root
    if l < n and arr[l] < arr[smallest]:
        smallest = l

    # If right child exists and is smaller than smallest so far
    if r < n and arr[r] < arr[smallest]:
        smallest = r

    # If smallest is not root,
    # swap and continue heapifying
    if smallest != i:
        arr[i], arr[smallest] = arr[smallest], arr[i]
        # Recursively heapify
        heapify(arr, smallest, n)

def sort(list_of_lists):
    # (value, list_idx, element_idx)
    arr = []
    for i, lst in enumerate(list_of_lists):
        if lst:
            arr.append((lst[0], i, 0))

    # Build min-heap
    for i in range(len(arr) // 2 - 1, -1, -1):
        heapify(arr, i, len(arr))
    result = []
    while arr:
        val, list_idx, elem_idx = arr[0]
        result.append(val)

        next_idx = elem_idx + 1
        if next_idx < len(list_of_lists[list_idx]):
            arr[0] = (list_of_lists[list_idx][next_idx], list_idx, next_idx)
        else:
            arr[0] = arr[-1]
            arr.pop()
        # heapify
        heapify(arr, 0, len(arr))

    return result

a = [1, 5, 7]
b = [2, 4]
c = [3, 6, 8, 9]

list_of_lists = [a, b, c]
sorted_list = sort(list_of_lists)
print(sorted_list)

```

[1, 2, 3, 4, 5, 6, 7, 8, 9]

Exercise 5

a

When all elements in the array are the same, q from Partition returns the r.

```

1: Partition(A, p, r)
2: x = A[r]
3: i = p - 1
4: for j = p to r - 1
5:   if A[j] ≤ x
6:     i = i + 1
7:   swap A[i] and A[j]
8: swap A[i + 1] and A[r]
9: return i + 1

```

Step 5, since all elements are the same, A[j] will always be = x. Then i will always be incremented in step 6. After the loop, i will be r-1. Then in step 9, it will return i+1, i.e. r

b

This is the worst case for the algorithm. Because it will need to do the splitting n times. With always-unbalanced splits we get:

$$T(n) = T(n-1) + \Theta(n) \dots \dots > T(n) = \Theta(n^2)$$

c

```

1: Partition(A, p, r)
2: if min(A[p..r]) = max(A[p..r])
3:   return [(p + r) / 2]
4: x = A[r]
5: i = p - 1
6: for j = p to r - 1
7:   if A[j] ≤ x
8:     i = i + 1
9:   swap A[i] and A[j]
10: swap A[i + 1] and A[r]
11: return i + 1

```

Exercise 6

a

If the quicksort could always partition on the median of the data in each subquery. This would be the optimal runtime case, where we get:

$$T(n) = \Theta(n \lg(n))$$

We don't think it can be implemented efficiently. We could have used Randomized-Select to find the median, and have used that median for partitioning. But we would still have ended up with a runtime of:

$$T(n) = \Theta(n \lg(n))$$

but with an larger cost per step, that would in practice make it slower.